Force-Test Tester Landing Page — Vendor Integration Guide

For: the web vendor building the customer landing pages on force-test.com **API provider:** QualityTester.us (Force-Test) **Version:** 1.0 — July 14, 2026

1. What you are building

Every Force-Test tester ships with a QR code printed on its calibration envelope. The QR encodes a URL of the form:

```
https://force-test.com/<serial-number>        e.g. https://force-test.com/26080
```

Your landing page at that URL must offer, for that specific serial number:

1. **User manual** — rendered in HTML on the page, plus a "Download PDF" button.
2. **Product registration** — a form collecting company and contact information.
3. **Calibration certificate** — a viewer/download for the latest calibration certificate PDF.

All tester data comes from the QualityTester API described below. You never receive a database export; the page calls the API live.

2. API basics

Base URL	`https://qualitytester.us/api/v1`
Auth	Header `X-API-Key: <vendor key>` on every request (provided separately, via a secure channel)
Transport	HTTPS only. Plain HTTP is redirected and must never be used.
CORS	Browser calls are allowed only from `https://force-test.com` and `https://www.force-test.com`
Errors	JSON: `{"error": "message"}` with conventional status codes (400, 401, 404, 429)
Rate limits	120 reads/min/IP, 60 certificate downloads/min/IP, 10 registrations/hour/IP. On 429, honor `Retry-After`.

The vendor key identifies your site and enables abuse controls. It is not a customer secret, but do not publish it outside the landing-page code and do not use it from any other domain. If it is ever compromised, notify Force-Test and a replacement will be issued (the old key stops working).

3. Endpoints

3.1 Tester lookup

```
GET /api/v1/testers/{serial}
```

Response `200`:

```
{
  "serial": "26080",
  "model": "SP1-2KD",
  "gage_type": "digital",
  "capacity_lbs": 2000,
  "last_cal_date": "2026-07-01",
  "cal_due_date": "2027-07-01",
  "registered": false,
  "has_certificate": true,
  "certificate_date": "2026-07-01",
  "manual": {
    "html_url": "/api/v1/testers/26080/manual",
    "pdf_url": "/api/v1/testers/26080/manual.pdf"
  },
  "certificate_pdf_url": "/api/v1/testers/26080/certificate.pdf"
}
```

`404` means the serial is unknown — show a friendly "tester not found, contact Force-Test" page. Null `html_url`/`pdf_url`/`certificate_pdf_url` mean that item is not yet available for this tester; hide or disable the corresponding section.

3.2 User manual

```
GET /api/v1/testers/{serial}/manual        -> text/html
GET /api/v1/testers/{serial}/manual.pdf    -> application/pdf
```

The HTML body is self-contained (inline styles). Render it inside a sandboxed container.

3.3 Latest calibration certificate

```
GET /api/v1/testers/{serial}/certificate.pdf   -> application/pdf
```

Always returns the most recent certificate for that serial.

3.4 Product registration

```
POST /api/v1/testers/{serial}/register
Content-Type: application/json
```

Body fields (all strings, max 300 chars each):

Field	Required
`company`	yes
`contact_name`	yes
`email`	yes (validated)
`phone`, `address`, `city`, `state`, `zip`, `country`, `purchase_date`	optional

Success: `201 {"ok": true, "registration_id": 123}`. Validation problems return `400 {"error": "..."}` — surface the message next to the form.

4. Sample code

Because every request needs the `X-API-Key` header, PDFs cannot be plain `<a href>` / `<iframe src>` links. Fetch them as blobs:

```
<script>
const API = "https://qualitytester.us/api/v1";
const KEY = "YOUR_VENDOR_KEY";
```

```
const serial = decodeURIComponent(location.pathname.replace(/^\//, "")); // force-test.com/26080

async function api(path, opts = {}) {
  const r = await fetch(API + path, {
    ...opts,
    headers: { "X-API-Key": KEY, ...(opts.headers || {}) }
  });
  if (!r.ok) throw Object.assign(new Error("api"), { status: r.status, body: await r.text() });
  return r;
}

async function load() {
  const info = await (await api(`/testers/${encodeURIComponent(serial)}`)).json();

  document.getElementById("title").textContent =
    `Force-Test ${info.model} — Serial ${info.serial}`;

  if (info.manual.html_url) {
    document.getElementById("manual").innerHTML =
      await (await api(`/testers/${serial}/manual`)).text();
  }

  if (info.certificate_pdf_url) {
    const pdf = await (await api(`/testers/${serial}/certificate.pdf`)).blob();
    const url = URL.createObjectURL(pdf);
    document.getElementById("cert-frame").src = url;        // <iframe id="cert-frame">
    document.getElementById("cert-dl").href = url;          // <a download="calibration.pdf">
  }
}

async function register(formData) {
  const r = await api(`/testers/${serial}/register`, {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify(Object.fromEntries(formData))
  });
  return r.json();   // {ok: true, registration_id: n}
}
load();
</script>
```

Manual PDF download works the same way as the certificate (fetch → blob → object URL).

5. Requirements and constraints

- **HTTPS everywhere.** The landing page must be served over HTTPS or browser CORS will fail.
- **Do not store or log registration data** on your side beyond the in-flight request; the API is the system of record and the data is encrypted at rest there.
- **Do not cache certificate or registration responses** in any shared cache/CDN (the API already sends `Cache-Control: no-store`). Manual HTML/PDF may be cached client-side.
- **Serial comes only from the URL path.** Do not build serial pickers, search, or enumeration UI — one QR scan, one serial, one page.
- **Handle 429** by backing off; do not retry in a loop.
- The API never exposes customer data: `registered` is a boolean only. There is no endpoint to read back registrations.

6. Support

Key issuance/rotation and test serial numbers for development: contact Force-Test, (727) 421-3093. A dev serial with a manual and sample certificate will be provisioned on request so you can build against real responses.